

CliMATE — Weather App

June 2026 · Front-End Web App · React · Vite · Material UI

Anshumaan Sai Patnaik

Overview

CliMATE is a fast, responsive weather application that shows real-time conditions for any city, town, or landmark. The user types a location, and the app returns the current temperature along with humidity, pressure, wind speed, and visibility in a clean Material UI card. It is a front-end React application that talks directly to two external APIs.

The interesting part is not the UI alone but the lookup pipeline behind it: a free-text place name has to be turned into map coordinates before any weather service can be queried. CliMATE handles that in two chained steps and gracefully covers the cases where a place can't be found.

Technology Stack

React	Component-based UI with local state driving what is shown.
Vite	Build tool and fast development server.
Material UI	Styled input components, alongside custom CSS for the weather card.
Geoapify API	Geocoding — converts a typed location into latitude/longitude.
OpenWeather API	Returns current weather data for those coordinates (metric units).
Vite env vars	API keys are read from environment variables rather than hard-coded.

Architecture & Key Logic

The app is built from small, single-purpose components. **App** holds the one piece of state that matters — the current weather result — and passes a setter down to the search form. **SearchForm** captures the query and runs the lookup; the result is then rendered by **WeatherCard**, which splits into a **Header** (location and temperature) and a **Stats** row (humidity, pressure, wind, visibility), with a dedicated **EmptyWeatherCard** for the not-found state.

The two-step lookup pipeline

Weather APIs need coordinates, not place names, so a search can't go straight to OpenWeather. CliMATE first sends the typed text to Geoapify, reads the latitude and longitude off the best match, and only then queries OpenWeather with those coordinates. Each API call lives in its own module and throws on a failed response, so the form can handle errors in one place.

```
// Two-step lookup: address -> coordinates (Geoapify) -> weather (OpenWeather)
const geoCodeData = await geoCodeAddress(formInput);

if (geoCodeData.features.length > 0) {
  const { lat, lon } = geoCodeData.features[0].properties;
  weatherData = await getWeather(lat, lon);
} else {
  weatherData = "No Data"; // no match -> show the 404 card
}

setWeatherInfo(weatherData);
```

Rendering the result state

What appears on screen is driven entirely by that single state value. While it is empty, no card shows; when the lookup returns "No Data", the app renders the EmptyWeatherCard with a "404 Location" message; otherwise it renders the full weather card. This keeps the three possible states — nothing searched, not found, and found — clearly separated.

Shaping the data & UX details

The raw API response is tidied up at the point of display: the weather description is capitalised, wind speed is converted from metres per second to kilometres per hour, and visibility is shown in kilometres. Small touches round out the experience — submitting an empty field replays a "wiggle" animation on the input to draw attention to it, and the field blurs on submit so pressing Enter behaves exactly like clicking the search button.